TAPZA TECHNOLOGIES · MOBILE ENGINEER

Tapza Care — React Native Assignment

Time limit: 48 hours from the moment this is shared **Stack:** React Native (Expo allowed), TypeScript

Backend: mock it

Objective

Tapza Clinic is clinic software used by doctors in India. Patients receive their prescription on WhatsApp after a visit. **Tapza Care** is the patient app: prescriptions, medicine reminders, bookings, and, soon, ordering clinic services the way people order groceries on Zepto or Blinkit.

Build the first slice of that app: a **Zepto-style home screen** whose sections appear as you scroll, each with its own background, all driven by a remote config an admin could change for a festival without a release; plus a booking flow and a prescriptions screen.

No Figma is provided. You design it. The audience is Indian patients and their families on mid-range Android phones, many of them older, many who prefer Hindi or Telugu.

What we are looking for

Creativity and a feel for UI/UX

Can you make something calm, trustworthy and pleasant to use without a designer beside you?

Passion for detail

Smooth animations, considered spacing, loading and empty states that look designed, not bolted on.

Speed with understanding

Use any AI agent or tool you like. You must be able to explain every file and change it live.

Clean code and structure

Folder structure that another engineer can navigate, strict TypeScript, no hard-coded colours.

Functional Requirements

1. Home (Zepto-style, config-driven)

The app fetches `GET /config` and renders whatever it describes: theme colours, tabs, and an ordered list of sections. Each section carries its **own background** (solid colour, gradient or image) so the home reads as distinct bands as you scroll. Sections animate in as they enter the viewport; the header shrinks or changes as you scroll. Support at least these section types:

`hero_banner`	full-width image or gradient with greeting, parallax on scroll
`category_chips`	horizontal scrollable chips (Consult, Lab tests, Pharmacy, Vaccines…)
`quick_actions`	icon tiles: Book, Prescriptions, Reminders, Family

`service_grid`	2 or 3 column cards with image, name, price in rupees, optional badge
`doctor_carousel`	horizontal cards with photo, specialty, next available slot
`offer_strip`	thin coloured band with a message

Ship **two configs** — a normal day and a festival day (Diwali, Pongal, Eid, your pick) — and a way to switch at runtime. Switching must change colours, banner, section order and items with **no code change**. An unknown section type must render nothing and never crash.

2. Booking flow

From a doctor or service: pick a date, pick a slot in a **gesture-driven bottom sheet**, confirm, see a success animation. Show loading, empty and error states. Confirming should feel instant (optimistic update, then reconcile with the mock).

3. Prescriptions and reminders

List prescriptions, open one, show its medicines as a morning / afternoon / night schedule, and let the user mark a dose as taken.

4. Resilience and states

Your mock must have a switch for **latency** and **failure**. Slow network, a failed config fetch, and an empty list must all leave the app usable. Cache the last good config. Skeleton loaders should match the shape of the real content.

Mock backend

Mock it any way you like (bundled JSON with simulated delay, MSW, json-server). Use these endpoints and shapes so submissions are comparable; invent realistic Indian content (names, a Hyderabad or Vijayawada clinic, prices in ₹).

```
GET  /config                            → LayoutConfig
GET  /doctors                           → Doctor[]
GET  /slots?doctorId=&date=YYYY-MM-DD   → Slot[]
POST /bookings {doctorId, slotId}       → Booking (201) · 409 if slot taken
GET  /prescriptions                     → Prescription[]

LayoutConfig = {
  version: number
  theme: { primary, secondary, background, surface, textPrimary,
           textSecondary, accent: string,
           festival?: { name, greeting, bannerImageUrl: string } }
  tabs: { id, label, icon, screen: 'home'|'bookings'|'prescriptions'|'profile' }[]
  sections: { id: string, type: string,
              background: { kind: 'color'|'gradient'|'image', value: string },
              title?: string, items: unknown[] }[]      // unknown type → skip
}
Doctor = { id, name, specialty, photoUrl, feeInr, languages: string[] }
Slot   = { id, doctorId, startsAt, endsAt, available: boolean }
Prescription = { id, issuedAt, doctorName, clinicName,
                 medicines: { id, name, dose, days,
                              timing: ('morning'|'afternoon'|'night')[] }[] }
```

Tech and rules

- **React Native** is mandatory (Expo or bare, your choice). Everything else is your call: navigation, state, data fetching, animation library, styling approach.
- **TypeScript strict.** Colours, spacing and type sizes come from a theme derived from the config; nothing hard-coded in components.
- **Animations must run smoothly** on a mid-range Android device. Lists must not stutter.
- **AI agents are welcome** (Claude Code, Cursor, Copilot, anything). Keep an `AI_LOG.md` with the prompts that shaped the code and what you rejected. In the follow-up call we will ask you to change things live; owning your code matters more than who typed it.
- Accessibility basics: OS font scaling respected, tap targets at least 44 pt, readable contrast in both themes.
- A few meaningful tests: config validation, one section renderer, booking state.

Bonus (optional, extra points)

- Local notifications for medicine reminders
- Hindi or Telugu behind an i18n layer
- Dark mode driven by the config
- Shared-element transition into detail screens
- Haptic feedback on key actions
- Offline queue: book while offline, sync later
- Deep links into a doctor or prescription
- A small in-app “admin” screen to edit the config live
- End-to-end test (Detox or Maestro)
- Lottie or custom success animation

Submission

- **GitHub repository** with a meaningful commit history across the 48 hours, not one commit at the end. Share the repository link with us when you submit.
- **README**: how to run, folder structure and why, architecture decisions, what you cut and why, honest time spent.
- **Screenshots and a 2–3 minute demo video** in the repo under `/docs`, showing both themes, the booking flow with a forced failure, and reminders. Record on a real Android device if you can.
- **Installable build**: attach the APK to a GitHub Release (or share an Expo build link) and put the download and install steps in the README.
- `AI_LOG.md` as described above.

A polished three-screen app beats an unfinished six-screen one. Tell us what you left out. Questions during the assignment are welcome; good questions count in your favour.

Time Limit

48 hours from the time the assignment is shared.